

Informe: mitigación de saturación Disk IO / carga de base de datos

Sistema: El Pulpo (POS multi-sucursal)
Fecha del informe: 11 de agosto de 2026
Proyecto Supabase activo (referencia): `apmsuigcveqtjzbpfihb` (plan Free Nano)
Contexto operativo del incidente: ~~3 sucursales con turno OPEN, ~3 dispositivos por sucursal (9 tablets)~~

1. Resumen ejecutivo

Se diagnosticó y se mitigó una saturación de **Disk IO** y consultas lentas/bloqueadas en Supabase, con síntomas en UI (alias parpadeando a “Usuario”, menú operativo desapareciendo, lentitud general, y más adelante regresiones en Despachar/Cobrar por cambios agresivos en RPCs).

La base de datos es **pequeña** (~55 MB), pero estaba **sobre-consultada** (millones de sequential scans en tablas chicas; hotspots en Realtime, gate de turno, cobro/despacho y lecturas de `cash_shifts`).

Enfoque final adoptado: reducir presión desde el cliente + índices aditivos + mitigaciones SQL de cobro/despacho **revertidas** a versiones estables tras regresiones. No se depende de un upgrade a Pro para “arreglar lógica”; Pro solo ampliaría el techo de IO si el pico sigue excediendo Free Nano.

2. Problema reportado (síntomas)

Síntoma	Descripción
Auth / menú	Alias (ej. WILL) parpadeaba a “ Usuario ”; el menú operativo desaparecía y luego volvía
Rendimiento	Sistema muy lento con varios turnos y tablets abiertos
Dashboard Supabase	Disk IO Budget depleting; Exhausting resources; Disk IO alto; Slow queries; DB errors; queries bloqueadas con <code>UPDATE orders</code>
Post-cambios (regresión)	Despachar / Despachar todo: ítem desaparecía y volvía
Post-cambios (regresión)	Cobrar: botón sin efecto aparente

3. Diagnóstico (causas raíz)

3.1 Infraestructura y carga

- Plan **Free Nano** con techo bajo de Disk IO / CPU.
- BD pequeña pero con **patrón de acceso agresivo** (refetch, Realtime, RPCs frecuentes desde muchas tablets).

3.2 UI “Usuario” / menú

- `getUserDisplayName` devolvía “Usuario” si `profile` era `null` .
- Fallos de lectura vía helpers que devolvían `[]` borraban estado usable.

- El menú exige `shiftOpen && userEnabled` (gate de turno); un gate vacío/transitorio ocultaba el menú.

3.3 Presión de locks / cobro-despacho

- Snapshots operativos y sync de pago bajo `FOR UPDATE` en `orders` .
- Posible N+1 de snapshot en despacho.
- `compact_table_order_positions` en el camino crítico de sync (locks cruzados entre órdenes de la misma mesa).
- Cliente: invalidación Realtime demasiado amplia; polls/safety agresivos; repair de turno y lecturas de `cash_shifts` en cada refresh.

4. Cambios realizados

4.1 Resiliencia Auth / Branch (cliente)

Área	Cambio
AuthContext	Perfil por select directo (no <code>dbSelect</code>); no borrar perfil en error; no refetch de perfil en <code>TOKEN_REFRESHED</code>
BranchContext	Depende de <code>userId</code> ; no sobrescribir acceso usable con payload vacío
Sesión	Poll de validación de sesión única subido a 5 min (<code>AUTH_SESSION_POLL_MS</code>)

Objetivo: evitar parpadeo a “Usuario” y pérdida temporal del menú ante fallos/red/refresco de token.

4.2 Menos presión IO — cliente (Realtime, polls, invalidaciones)

Archivo central: `src/lib/queryEgress.ts` y `hooks/páginas operativas`.

Parámetro / comportamiento	Antes (aprox.)	Después (vigente)
Debounce hub Realtime	2,5 s	3 s
Backup listas si hub $\neq$ SUBSCRIBED	45 s	60 s
Safety Despacho/Servir (hub sano)	30 s	60 s
Safety global listas	0 (sin poll)	0 (sin cambio)
Stale listas operativas	15–20 s	25 s
Gate de turno stale	30 s	60 s
Auth session poll	3 min	5 min

Otros:

- **Invalidación Realtime selectiva** por tipo de evento (`order_items` , `orders` , `ready` , `dispatch` , `payments` , `shift`) vía `keysForHubSource` .
- `order_items` **ya no** invalida todas las colas de canal (Extra/Express/...); bastan cocina/despacho/mesas + prefijo de orden.
- Alertas mesero (`get_mesero_ready_alerts`) **desmontadas** de `AppLayout` (presión Disk IO).
- Despacho/Servir/Cocina: sin `refetchOnMount: "always"` ; Despacho/Servir sin `refetchOnWindowFocus` .

- Extra / Express / Para llevar / Orden especial: **sin suscripción Realtime a payments** (el estado de cobro llega por cambios en `orders`). Caja, Despacho/Servir y Mesas **sí** siguen escuchando pagos.
- Mesas: stale **5 s** → **25 s**.

Objetivo: menos refetch en segundo plano con 9 tablets, sin cambiar reglas de negocio de cobro/despacho.

4.3 Gate / turno en memoria (cliente)

Cambio	Detalle
<code>openCashShiftFromGate</code>	Helper en <code>src/lib/openCashShift.ts</code> : usa <code>shiftId + openedAt</code> del gate; no inventa <code>opened_at: ""</code> (eso rompía el filtro de turno)
Uso	Caja, Cocina, Órdenes (<code>useOrdersByStatus</code>), Despacho/Servir, Mesas
Repair de turno	Throttle 2 min → 5 min ; en Despacho/Servir el repair no bloquea el listado (como Cocina); solo se fuerza si la cola viene vacía

Objetivo: menos lecturas a `cash_shifts` y menos RPC `repair_open_shift_order_cash_shift_ids` en cada poll.

4.4 Base de datos — migraciones

Orden cronológico y estado operativo:

Migración	Contenido	Estado / nota
<code>20260811123000_reduce_order_lock_pressure.sql</code>	Snapshot unitario filtrado por ítems de la orden; <code>register_payment_with_items</code> con sync una vez al final (GUC <code>app.skip_payment_state_sync</code>)	Mitigación válida de locks
<code>20260811140000_defer_compact_and_dispatch_snapshot.sql</code>	Reescritura agresiva de despacho (temp tables, skip sync si PAID, compact diferido) + cobro con cola diferida	Problemática: regresiones Despachar/Cobrar
<code>20260811150000_sync_single_snapshot_and_hotspot_indexes.sql</code>	Sync con snapshot materializado + índices hotspots	Incluye rewrite de sync; índices cubiertos también por ...170000
<code>20260811160000_restore_stable_dispatch_order_quantities.sql</code>	HOTFIX: restaura <code>dispatch_order_quantities</code> estable; compact diferido desactivado (compact inmediato)	Obligatorio si se aplicó ...140000
<code>20260811161000_restore_stable_register_payment.sql</code>	HOTFIX: restaura <code>register_payment_with_items</code>	Obligatorio si se aplicó ...140000

	estable (sin cola diferida). Corregido BOM UTF-8 que impedía db push	
20260811170000_hotspot_indexes_only.sql	Solo índices IF NOT EXISTS (gate, eventos APPLIED, joins ítem→evento, mesa activa, order_items/payments)	Seguro; no toca RPCs

Índices incluidos (seguro / idempotente)

- idx_cash_shift_users_shift_user
- idx_cash_shift_users_shift_caja_enabled
- idx_order_ready_events_order_applied
- idx_order_dispatch_events_order_applied
- idx_order_cancellations_order_applied
- idx_order_item_ready_events_ready_event_id
- idx_order_item_dispatch_events_dispatch_event_id
- idx_order_item_cancellations_cancellation_id
- idx_orders_table_active_position
- idx_order_items_order_id_status
- idx_payments_order_id_status

4.5 Correcciones de UX ligadas a las regresiones

Problema	Corrección
Despachar: desaparece y vuelve	Hotfix SQL de despacho estable; se revirtió el “parche” optimista de epoch/cache que empeoraba el parpadeo
Cobrar: no hace nada	Hotfix SQL de cobro estable; PaymentDialogV2 ahora muestra toast si no puede cobrar y reconstruye cantidades pendientes si el mapa no estaba hidratado
db push falla en ...61000	Eliminado BOM UTF-8 (EF BB BF) del archivo SQL

4.6 Documentación actualizada

- docs/system_context.md
- docs/database_architecture.md
- docs/codex_rules.md
- docs/operational-module-refresh.md
- (referencias en arquitectura / auditoría de rendimiento según corresponda)

5. Lecciones aprendidas

1. **No reescribir RPCs de cobro/despacho** en el mismo ciclo que la saturación de IO sin staging y prueba manual.
2. Cambios **aditivos** (índices, cliente, throttle) son preferibles a compact diferido / temp tables en el camino crítico.
3. Un archivo SQL con **BOM** rompe supabase db push con error críptico en la línea de comentario.

4. Usar `opened_at: ""` desde el gate **rompe** el filtro de pertenencia al turno; siempre exigir `openedAt` real.

6. Estado recomendado en remoto

Aplicar / confirmar, en este orden conceptual:

1. Hotfixes de estabilidad: `...160000` , `...61000` (si alguna vez se aplicó `...140000`).
2. Cobro con sync única: `...123000` (si aún no estaba).
3. Índices seguros: `...170000` (y/o índices de `...150000` si esa migración ya corrió).
4. **Desplegar frontend** con todos los afinados de cliente descritos arriba.
5. Medir 20–30 min en pico: Disk IO, slow queries, blocked `UPDATE orders` .

No reaplicar el rewrite agresivo de despacho/cobro de `...140000` sin acuerdo explícito y prueba.

7. Qué queda fuera (consciente)

- Upgrade Free Nano → Pro: **no es requisito de lógica**, pero sigue siendo el techo de IO si el pico supera el plan.
- No se reintrodujeron alertas mesero globales.
- No se volvió a implementar despacho con snapshot único / compact diferido tras la regresión.

8. Criterios de éxito

- Alias y menú estables (sin parpadeo a “Usuario” / menú vacío).
- Cobrar y Despachar confiables (sin desaparecer-y-volver / botón muerto).
- Menor Disk IO y menos slow/blocked queries en hora pico con ~9 tablets.
- UX operativa aceptable en Free Nano tras deploy de cliente + índices.

9. Próximos pasos sugeridos

1. Validar métricas post-deploy (dashboard Supabase).
2. Si Disk IO sigue alto: solo afinados de cliente adicionales (sin tocar RPCs críticos).
3. Si el plan Free se agota pese a código estable: evaluar Pro como capacidad de infraestructura, no como “parche de bugs”.

Informe generado a partir del trabajo de mitigación del 11 de agosto de 2026 en el repositorio `sistema-el-pulpo` .